
TRÍ TUỆ NHÂN TẠO

Linear Regression (Taxi Fare)

Bài tập Colab — Dự đoán giá cước taxi (dịch tiếng Việt)

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Phần 1 — Chuẩn bị	4
1.1 Nạp các module cần thiết	4
1.2 Nạp dataset	4
1.3 Chọn cột cần dùng	5
2 Phần 2 — Khám phá dataset	6
2.1 Xem thống kê mô tả	6
2.2 Ma trận tương quan (correlation matrix)	7
2.3 Trục quan hóa quan hệ bằng pair plot	8
3 Phần 3 — Huấn luyện mô hình	9
3.1 Định nghĩa các hàm tạo và huấn luyện mô hình	9
3.2 Huấn luyện mô hình với một feature	10
3.3 Thử nghiệm siêu tham số	11
3.4 Huấn luyện mô hình với hai feature	13
3.5 So sánh các experiment	14
4 Phần 4 — Kiểm tra mô hình (validation)	15
4.1 Dùng mô hình để dự đoán	15

Lời nói đầu

Nguồn

Tài liệu dịch từ notebook Colab *Linear Regression* (ML Crash Course, 2023): https://colab.research.google.com/github/google/eng-edu/blob/main/ml/cc/exercises/linear_regression_taxi.ipynb.

Mục tiêu bài tập

Bạn sẽ dùng một dataset thật để huấn luyện mô hình dự đoán **giá cước** của một chuyến taxi ở Chicago (Illinois, Hoa Kỳ).

Mục tiêu học tập

Sau khi làm xong, bạn có thể:

- Đọc file `.csv` vào pandas DataFrame.
- Khám phá dataset bằng các thư viện trực quan hóa.
- Thử nhiều lựa chọn đặc trưng (features) để xây mô hình hồi quy tuyến tính.
- Tinh chỉnh siêu tham số (hyperparameters) của mô hình.
- So sánh các lần chạy qua RMSE và loss curve.

Mô tả dataset

Dataset cho bài tập: https://download.mlcc.google.com/mledu-datasets/chicago_taxi_train.csv là một phần con của bộ dữ liệu *City of Chicago Taxi Trips*: <https://data.cityofchicago.org/Transportation/Taxi-Trips/wrvz-psew>. Dữ liệu tập trung vào một giai đoạn 2 ngày trong tháng 5/2022.

1 Phần 1 — Chuẩn bị

1.1 Nạp các module cần thiết

Vì sao cần nhiều thư viện?

Bài tập sử dụng nhiều thư viện Python để thao tác dữ liệu, huấn luyện mô hình, và trực quan hóa.

Hướng dẫn

1. Chạy cell **Install required libraries**.
2. Chạy cell **Load dependencies**.

```
#@title Install required libraries

!pip install google-ml-edu==0.1.3 \
  keras~=3.8.0 \
  matplotlib~=3.10.0 \
  numpy~=2.0.0 \
  pandas~=2.2.0 \
  tensorflow~=2.18.0

print('\n\nAll requirements successfully installed.')
```

```
#@title Code - Load dependencies

# data
import numpy as np
import pandas as pd

# machine learning
import keras
import ml_edu.experiment
import ml_edu.results

# data visualization
import plotly.express as px
```

1.2 Nạp dataset

Đọc CSV vào DataFrame

Cell sau sẽ nạp dataset và tạo một pandas DataFrame. Bạn có thể coi DataFrame giống như một bảng tính (spreadsheet) có hàng và cột: hàng là các mẫu dữ liệu, cột là các thuộc tính của mỗi mẫu.

```
# @title
```

```
chicago_taxi_dataset = pd.read_csv("https://download.mlcc.google.com/mledu-  
datasets/chicago_taxi_train.csv")
```

1.3 Chọn cột cần dùng

Giữ lại một số cột

Cell sau chỉ giữ lại một số cột cần thiết trong dataset.

```
#@title Code - Read dataset  
  
# Updates dataframe to use specific columns.  
training_df = chicago_taxi_dataset.loc[:, ('TRIP_MILES', 'TRIP_SECONDS', 'FARE', '  
COMPANY', 'PAYMENT_TYPE', 'TIP_RATE')]  
  
print('Read dataset completed successfully.')  
print('Total number of rows: {0}\n\n'.format(len(training_df.index)))  
training_df.head(200)
```

2 Phần 2 — Khám phá dataset

2.1 Xem thông kê mô tả

Vì sao phải “hiểu dữ liệu” trước?

Trong hầu hết dự án ML, một phần rất lớn là hiểu dữ liệu: phạm vi giá trị, outlier, phân phối, biến phân loại, thiếu dữ liệu, ...

Hướng dẫn

1. Chạy cell **View dataset statistics**.
2. Nhìn bảng và trả lời:
 - Giá cước tối đa là bao nhiêu?
 - Quãng đường trung bình cho mọi chuyến là bao nhiêu?
 - Có bao nhiêu hãng taxi trong dataset?
 - Loại thanh toán phổ biến nhất là gì?
 - Có cột nào bị thiếu dữ liệu không?
3. Chạy cell **View answers...** để kiểm tra.

Vì sao có NaN trong bảng describe?

Khi làm dữ liệu trong Python, bạn có thể thấy NaN (not a number) nếu một phép tính không thể thực hiện hoặc dữ liệu bị thiếu. Trong dataset taxi, `PAYMENT_TYPE` và `COMPANY` là biến phân loại (không phải số), nên các thống kê như mean/max không có nghĩa và được hiển thị là NaN.

```
#@title Code - View dataset statistics
```

```
print('Total number of rows: {0}\n\n'.format(len(training_df.index)))
training_df.describe(include='all')
```

Đáp án tham khảo (từ notebook)

```
#@title Double-click or run to view answers about dataset statistics
```

```
answer = '''
```

```
What is the maximum fare?
.25
```

Answer: \$159

```
What is the mean distance across all trips?
```

Answer: 8.2895 miles

```
How many cab companies are in the dataset?
```

Answer: 31

```
What is the most frequent payment type?
```

Answer: Credit Card

```
Are any features missing data?
```

Answer: No

```
'''
```

```
# What is the maximum fare?
max_fare = training_df['FARE'].max()
print("What is the maximum fare?
      {}".format(fare = max_fare))

# What is the mean distance across all trips?
mean_distance = training_df['TRIP_MILES'].mean()
print("What is the mean distance across all trips?
      miles".format(mean = mean_distance))

# How many cab companies are in the dataset?
num_unique_companies = training_df['COMPANY'].nunique()
print("How many cab companies are in the dataset?
      {}".format(number = num_unique_companies))

# What is the most frequent payment type?
most_freq_payment_type = training_df['PAYMENT_TYPE'].value_counts().idxmax()
print("What is the most frequent payment type?
      {}".format(type = most_freq_payment_type))

# Are any features missing data?
missing_values = training_df.isnull().sum().sum()
print("Are any features missing data?
      {}".format(missing_values == 0 else "Yes"))
```

2.2 Ma trận tương quan (correlation matrix)

Chọn feature nên dựa vào tương quan

Một bước quan trọng là xem feature nào tương quan tốt với **nhân** (FARE). Thông thường giá cước liên quan đến quãng đường và thời lượng chuyến đi. Ma trận tương quan giúp bạn định lượng điều này.

Ý nghĩa giá trị tương quan

- 1.0: tương quan dương hoàn hảo (một tăng thì cái kia tăng).
- -1.0: tương quan âm hoàn hảo (một tăng thì cái kia giảm).
- 0.0: không tương quan tuyến tính đáng kể.

Nhìn chung, $|\text{corr}|$ càng lớn thì sức dự đoán càng tốt.

Hướng dẫn kiểm tra hiểu

Chạy cell sau và trả lời:

- Feature nào tương quan mạnh nhất với FARE?
- Feature nào tương quan yếu nhất với FARE?

#@title Code - View correlation matrix

```
training_df.corr(numeric_only = True)
```

Đáp án tham khảo (từ notebook)

```
#@title Double-click to view answers about the correlation matrix

answer = '''
The feature with the strongest correlation to the FARE is TRIP_MILES.
As you might expect, TRIP_MILES looks like a good feature to start with to train
the model. Also, notice that the feature TRIP_SECONDS has a strong correlation
with fare too.
'''
print(answer)

answer = '''The feature with the weakest correlation to the FARE is TIP_RATE.'''
print(answer)
```

2.3 Trực quan hóa quan hệ bằng pair plot

Pair plot / scatter matrix

Chạy cell sau để xem scatter matrix giữa các cột quan trọng.

```
#@title Code - View pairplot
px.scatter_matrix(training_df, dimensions=["FARE", "TRIP_MILES", "TRIP_SECONDS"])
```


3 Phần 3 — Huấn luyện mô hình

3.1 Định nghĩa các hàm tạo và huấn luyện mô hình

Hướng dẫn

Chạy cell **Define ML functions**. Nếu muốn hiểu sâu, hãy mở rộng cell để đọc code.

```
#@title Code - Define ML functions

def create_model(
    settings: ml_edu.experiment.ExperimentSettings,
    metrics: list[keras.metrics.Metric],
) -> keras.Model:
    """Create and compile a simple linear regression model."""
    inputs = {name: keras.Input(shape=(1,), name=name) for name in settings.
        input_features}
    concatenated_inputs = keras.layers.Concatenate()(list(inputs.values()))
    outputs = keras.layers.Dense(units=1)(concatenated_inputs)
    model = keras.Model(inputs=inputs, outputs=outputs)

    model.compile(optimizer=keras.optimizers.RMSprop(learning_rate=settings.
        learning_rate),
        loss="mean_squared_error",
        metrics=metrics)

    return model

def train_model(
    experiment_name: str,
    model: keras.Model,
    dataset: pd.DataFrame,
    label_name: str,
    settings: ml_edu.experiment.ExperimentSettings,
) -> ml_edu.experiment.Experiment:
    """Train the model by feeding it data."""

    features = {name: dataset[name].values for name in settings.input_features}
    label = dataset[label_name].values
    history = model.fit(x=features,
        y=label,
        batch_size=settings.batch_size,
        epochs=settings.number_epochs)

    return ml_edu.experiment.Experiment(
        name=experiment_name,
        settings=settings,
        model=model,
        epochs=history.epoch,
        metrics_history=pd.DataFrame(history.history),
    )
```

```
print("SUCCESS: defining linear regression functions complete.")
```

3.2 Huấn luyện mô hình với một feature

Bắt đầu từ feature tương quan mạnh nhất

Ta bắt đầu với TRIP_MILES vì nó tương quan mạnh nhất với nhân FARE.

Hướng dẫn

1. Chạy cell **Experiment 1**.
2. Quan sát đường loss/RMSE và đồ thị dự đoán.
3. Trả lời:
 - Mất bao nhiêu epoch để mô hình hội tụ?
 - Mô hình khớp dữ liệu mẫu tốt đến đâu?

RMSE có đơn vị giống nhân

Trong huấn luyện, bạn sẽ thấy RMSE. Đơn vị RMSE giống đơn vị nhân (\$). RMSE giúp ước lượng dự đoán sai lệch trung bình bao nhiêu đô la so với giá quan sát.

```
#@title Code - Experiment 1

settings_1 = ml_edu.experiment.ExperimentSettings(
    learning_rate = 0.001,
    number_epochs = 20,
    batch_size = 50,
    input_features = ['TRIP_MILES']
)

metrics = [keras.metrics.RootMeanSquaredError(name='rmse')]

model_1 = create_model(settings_1, metrics)

experiment_1 = train_model('one_feature', model_1, training_df, 'FARE', settings_1
)

ml_edu.results.plot_experiment_metrics(experiment_1, ['rmse'])
ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')
```

Đáp án tham khảo (từ notebook)

```
#@title Double-click to view answers for training model with one feature

answer = ""
```

Use the loss curve to see where the loss begins to level off during training.

With this set of hyperparameters:

```
learning_rate = 0.001
epochs = 20
batch_size = 50

it takes about 5 epochs for the training run to converge to the final model.
"""
print(answer)

answer = '''
It appears from the model plot that the model fits the sample data fairly well.
'''
print(answer)
```

3.3 Thử nghiệm siêu tham số

Vì sao phải chạy nhiều experiment?

Trong ML, thường phải thử nhiều cấu hình siêu tham số để tìm bộ tốt nhất. Hãy thay đổi từng siêu tham số và quan sát loss curve và đồ thị mô hình.

Các thử nghiệm gợi ý

- **Experiment 1:** tăng learning rate lên 1.0 (giữ batch size 50).
- **Experiment 2:** giảm learning rate xuống 0.0001 (giữ batch size 50).
- **Experiment 3:** tăng batch size lên 500 (giữ learning rate 0.001).

Hướng dẫn

Sửa các siêu tham số trong cell **Experiment 2**, chạy, rồi quan sát khác biệt. Lập lại cho từng thử nghiệm. Trả lời:

- Tăng learning rate ảnh hưởng thế nào?
- Giảm learning rate ảnh hưởng thế nào?
- Thay batch size ảnh hưởng kết quả không?

```
#@title Code - Experiment 2
```

```
# The following variables are the hyperparameters.
# TODO - Adjust these hyperparameters to see how they impact a training run.
settings_2 = ml_edu.experiment.ExperimentSettings(
    learning_rate = 0.001,
    number_epochs = 20,
    batch_size = 50,
    input_features = ['TRIP_MILES']
```

```

)

metrics = [keras.metrics.RootMeanSquaredError(name='rmse')]

model_2 = create_model(settings_2, metrics)

experiment_2 = train_model('one_feature_hyper', model_2, training_df, 'FARE',
                             settings_2)

ml_edu.results.plot_experiment_metrics(experiment_2, ['rmse'])
ml_edu.results.plot_model_predictions(experiment_2, training_df, 'FARE')

```

Đáp án tham khảo (từ notebook)

```

#@title Double-click to view answers for hyperparameter experiments

answer = """
When the learning rate is too high, the loss curve bounces around and does not
appear to be moving towards convergence with each iteration. Also, notice that
the predicted model does not fit the data very well. With a learning rate that
is too high, it is unlikely that you will be able to train a model with good
results.
"""
print(answer)

answer = '''
When the learning rate is too small, it may take longer for the loss curve to
converge. With a small learning rate the loss curve decreases slowly, but does
not show a dramatic drop or leveling off. With a small learning rate you could
increase the number of epochs so that your model will eventually converge, but
it will take longer.
'''
print(answer)

answer = '''
Increasing the batch size makes each epoch run faster, but as with the smaller
learning rate, the model does not converge with just 20 epochs. If you have
time, try increasing the number of epochs and eventually you should see the
model converge.
'''
print(answer)

```

3.4 Huấn luyện mô hình với hai feature

Thêm feature có giúp tốt hơn không?

Mô hình dùng TRIP_MILES có sức dự đoán khá tốt, nhưng ta thử thêm feature thứ hai để cải thiện. Dataset không có TRIP_MINUTES sẵn, nhưng có thể suy ra từ TRIP_SECONDS.

Hướng dẫn

1. Xem code trong cell **Experiment 3**.
2. Chạy cell và so sánh RMSE với mô hình một feature.
3. Trả lời:
 - Mô hình 2 feature có tốt hơn không?
 - Dùng TRIP_SECONDS thay TRIP_MINUTES có khác không?
 - Mô hình có gần công thức tính giá thật không?

Vì sao có đồ thị 3D?

Scatter plot giữa (2 feature, 1 nhãn) là đồ thị 3D: hai feature trên trục x, y , nhãn trên trục z . Điểm dữ liệu là các vòng tròn, còn mô hình là một mặt phẳng. Nếu mô hình tốt, đa số điểm sẽ nằm gần mặt phẳng đó.

```
#@title Code - Experiment 3

settings_3 = ml_edu.experiment.ExperimentSettings(
    learning_rate = 0.001,
    number_epochs = 20,
    batch_size = 50,
    input_features = ['TRIP_MILES', 'TRIP_MINUTES']
)

training_df['TRIP_MINUTES'] = training_df['TRIP_SECONDS']/60

metrics = [keras.metrics.RootMeanSquaredError(name='rmse')]

model_3 = create_model(settings_3, metrics)

experiment_3 = train_model('two_features', model_3, training_df, 'FARE',
    settings_3)

ml_edu.results.plot_experiment_metrics(experiment_3, ['rmse'])
ml_edu.results.plot_model_predictions(experiment_3, training_df, 'FARE')
```

Đáp án tham khảo (từ notebook)

```
#@title Double-click to view answers for training with two features
```

```

answer = '''
To answer this question for your specific training runs, compare the RMSE for
each model. For example, if the RMSE for the model trained with one feature was
3.7457 and the RMSE for the model with two features is 3.4787, that means that
on average the model with two features makes predictions that are about $0.27
closer to the observed fare.

'''
print(answer)

answer = '''
When training a model with more than one feature, it is important that all
numeric values are roughly on the same scale. In this case, TRIP_SECONDS and
TRIP_MILES do not meet this criteria. The mean value for TRIP_MILES is 8.3 and
the mean for TRIP_SECONDS is 1,320; that is two orders of magnitude difference.
In contrast, the mean for TRIP_MINUTES is 22, which is more similar to the scale
of TRIP_MILES (8.3) than TRIP_SECONDS (1,320). Of course, this is not the
only way to scale values before training, but you will learn about that in
another module.
'''
print(answer)

answer = '''
In reality, Chicago taxi cabs use a documented formula to determine cab fares.
For a single passenger paying cash, the fare is calculated like this:

FARE = 2.25 * TRIP_MILES + 0.12 * TRIP_MINUTES + 3.25

Typically with machine learning problems you would not know the 'correct'
formula, but in this case you can use this knowledge to evaluate your model.
Take a look at your model output (the weights and bias) and determine how
well it matches the ground truth fare calculation. You should find that the
model is roughly close to this formula.
'''
print(answer)

```

3.5 So sánh các experiment

```

ml_edu.results.compare_experiment([experiment_1, experiment_3], ['rmse'],
    training_df, training_df['FARE'].values)

```

4 Phần 4 — Kiểm tra mô hình (validation)

4.1 Dùng mô hình để dự đoán

Hướng dẫn

1. Chạy cell **Define functions to make predictions**.
2. Chạy cell **Make predictions**.
3. Xem bảng dự đoán và trả lời: dự đoán gần nhãn thật đến mức nào?

```
#@title Code - Define functions to make predictions
def format_currency(x):
    return "${:.2f}".format(x)

def build_batch(df, batch_size):
    batch = df.sample(n=batch_size).copy()
    batch.set_index(np.arange(batch_size), inplace=True)
    return batch

def predict_fare(model, df, features, label, batch_size=50):
    batch = build_batch(df, batch_size)
    predicted_values = model.predict_on_batch(x={name: batch[name].values for name
        in features})

    data = {"PREDICTED_FARE": [], "OBSERVED_FARE": [], "L1_LOSS": [],
        features[0]: [], features[1]: []}
    for i in range(batch_size):
        predicted = predicted_values[i][0]
        observed = batch.at[i, label]
        data["PREDICTED_FARE"].append(format_currency(predicted))
        data["OBSERVED_FARE"].append(format_currency(observed))
        data["L1_LOSS"].append(format_currency(abs(observed - predicted)))
        data[features[0]].append(batch.at[i, features[0]])
        data[features[1]].append("${:.2f}".format(batch.at[i, features[1]]))

    output_df = pd.DataFrame(data)
    return output_df

def show_predictions(output):
    header = "-" * 80
    banner = header + "\n" + "|" + "PREDICTIONS".center(78) + "|" + "\n" + header
    print(banner)
    print(output)
    return
```

```
#@title Code - Make predictions

output = predict_fare(experiment_3.model, training_df, experiment_3.settings.
    input_features, 'FARE')
show_predictions(output)
```

Đáp án tham khảo (từ notebook)

```
#@title Double-click to view answers for validate model
```

```
answer = '''
Based on a random sampling of examples, the model seems to do pretty well
predicting the fare for a taxi ride. Most of the predicted values do not vary
significantly from the observed value. You should be able to see this by looking
at the column L1_LOSS = |observed - predicted|.
'''
print(answer)
```